Apple Orchard Monitoring System

End-to-End Deep Learning for Smart Agriculture

Master's Project Report

`https://github.com/Samer-Gassouma/Apple-Orchard-Monitor`

May 2026

1 Introduction

Apple cultivation is a significant agricultural sector, yet manual quality inspection remains labor-intensive, subjective, and error-prone. This project presents an end-to-end deep learning system for automated apple orchard monitoring that combines real-time object detection, instance segmentation, and explainable artificial intelligence (XAI).

The system was designed as a two-stage pipeline: a YOLOv8n detector identifies individual apples in orchard images, while a U-Net ResNet-34 model segments the precise boundary of each detected apple using high-quality masks generated by Meta's Segment Anything Model (SAM). A dual explainability layer provides human-interpretable justifications for model decisions, supported by quantitative validation metrics including faithfulness and cross-model alignment.

2 Dataset

Two apple datasets from Roboflow Universe were merged for training:

- `paramee/apple-tiyxx`: ∼216 images
- `l61l/apple-desti`: ∼1,270 images

The combined dataset provides bounding box annotations for apple detection. For segmentation training, high-quality masks were generated using Meta's Segment Anything Model (SAM) from the bounding box annotations, producing 1,040 annotated images with precise apple-shaped masks.

Training used an 85/15 train/validation split with images resized to 640×640 for YOLO and 256×256 for U-Net.

3 Methodology

3.1 Stage 1: Object Detection with YOLOv8n

YOLOv8n (nano variant) was selected for its lightweight architecture (3.2M parameters) and fast inference speed, making it suitable for real-time orchard monitoring. The model was initialized from the Ultralytics COCO pre-trained checkpoint and fine-tuned on the

merged apple dataset for 100 epochs with early stopping on an NVIDIA T4 GPU via Google Colab.

Training configuration:

- Image size: 640×640
- Batch size: 16
- Optimizer: SGD with momentum 0.937
- Learning rate: 0.01
- Augmentation: Mosaic, MixUp, random flip, HSV jitter

The model achieves excellent detection performance with mAP@0.5 of 0.987 and mAP@0.5:0.95 of 0.836 on the validation set.

3.2 Stage 2: Instance Segmentation with U-Net

A U-Net architecture with ResNet-34 encoder was implemented using the segmentation-models-pytorch library. The model outputs a single-channel binary mask (sigmoid activation) representing the apple foreground boundary.

Training configuration:

- Encoder: ResNet-34 (ImageNet pre-trained)
- Loss: Binary Cross-Entropy + Dice Loss
- Optimizer: AdamW with learning rate 10^{-3} and weight decay 10^{-4}
- Input: 256×256 cropped apple regions
- Epochs: 50 with early stopping (patience=10)
- Training data: 884 SAM-generated masks
- Validation data: 156 masks

Training converged in 14 epochs with a best validation loss of 0.324. The U-Net segments apple boundaries with high precision, enabling per-fruit coverage metrics and pixel-level analysis.

3.3 Stage 3: Explainability

Two complementary explainability methods were integrated:

EigenCAM is applied to the YOLO backbone to visualize which image regions influenced detection decisions. It computes the principal eigenvector of the feature covariance matrix without requiring gradient computation, making it stable for detection architectures.

Grad-CAM is applied to the U-Net decoder to explain segmentation decisions. It uses gradient information flowing into the final convolutional layer to produce a coarse localization map highlighting important regions for the segmentation output.

3.4 Stage 4: Validation Metrics

Faithfulness measures whether the explanation identifies pixels the model actually relies upon. The top 20% most salient Grad-CAM pixels are masked, inference is re-run, and the confidence drop is measured. A high score indicates the CAM correctly identifies decision-critical pixels.

Cross-model alignment measures whether YOLO (detection) and U-Net (segmentation) attend to the same image regions. The IoU between EigenCAM and Grad-CAM attention regions is computed as an agreement score.

4 System Implementation

The system is deployed as a web application with three layers:

Backend: FastAPI provides REST endpoints for detection, segmentation, full analysis, and explainability. Models are loaded once during application startup via a lifespan context manager, ensuring efficient inference.

Frontend: Streamlit provides an interactive interface with image upload, confidence threshold and mask opacity sliders, an explanation toggle, side-by-side original and annotated display, per-fruit detailed analysis panels, and annotated image download.

Configuration: All parameters (model paths, thresholds, image sizes, class names, colors, XAI settings) are centralized in a single `config.yaml` file, enabling reproducible experiments without code modification.

Containerization: A Dockerfile enables single-command deployment:

```
docker build -t apple-monitor .
docker run -p 8000:8000 apple-monitor
```

5 Interface Screenshots

The Streamlit frontend demonstrates the complete pipeline on a real orchard image. Screenshots are available in the project repository at `https://github.com/Samer-Gassouma/Apple-Orchard-Monitor` under `assets/screenshots/`.

Step 1 – Upload & Detect: The user uploads an orchard image via the Streamlit interface. YOLOv8n detects 6 apples and draws bounding boxes with confidence scores. The main dashboard shows the original image, annotated detections, and summary statistics (Total Fruits, Healthy, Defective counts).

Step 2 – Per-Apple Segmentation & Explainability: Clicking any detected apple reveals a detailed panel with:

- Detection metadata (class, confidence, bbox coordinates)
- U-Net ResNet-34 segmentation mask and fruit coverage (e.g., 0.784)
- YOLO EigenCAM full-image attention map
- U-Net Grad-CAM per-crop boundary attention map
- Faithfulness score (0.11–0.27) and CAM alignment (IoU)

Step 3 – Session Summary: The dashboard reports total fruits detected (6) and average fruit coverage (0.796) across the entire image.

All screenshots and source code are available at: `https://github.com/Samer-Gassouma/Apple-Orchard-Monitor`

6 Results and Discussion

6.1 Detection Performance

The YOLOv8n detector achieves excellent in-distribution performance with mAP@0.5 of 0.987 and mAP@0.5:0.95 of 0.836. On a held-out test set of 30 COCO apple images, the model detected 33 apples with high confidence, demonstrating robust generalization to orchard scenes.

6.2 Segmentation Performance

The U-Net trained on SAM-generated masks converged efficiently in 14 epochs with a best validation loss of 0.324. The model produces smooth, apple-shaped masks that accurately capture fruit boundaries, enabling precise per-fruit coverage analysis.

6.3 Explainability Validation

Quantitative XAI metrics were computed over 33 detected fruits:

Metric	**Mean**	**Std**
Faithfulness	0.302	0.210
Confidence Drop	0.146	–
CAM Alignment (IoU)	0.015	0.030

Table 1: XAI validation metrics on test set ($n = 33$ fruits).

The faithfulness score of 0.302 indicates that Grad-CAM identifies genuinely decision-critical pixels. The low cross-model alignment (0.015) is expected because YOLO's Eigen-CAM highlights "where is the apple" while U-Net's Grad-CAM highlights "what is the boundary" — complementary but spatially different tasks.

6.4 Inference Speed Benchmarks

End-to-end latency was measured on CPU (Intel i5):

Pipeline Stage	**Latency**	**Throughput**
YOLO Detection only	∼82 ms	12.2 FPS
Detection + Segmentation	∼107 ms	9.3 FPS
Full Pipeline (+XAI)	∼68 ms	14.7 FPS

Table 2: Inference speed benchmarks on CPU.

The lightweight YOLOv8n architecture enables real-time processing suitable for orchard deployment.

6.5 Limitations

The system exhibits known sensitivity to out-of-distribution images. Performance may degrade on photos from different geographic regions, camera equipment, or lighting conditions. This is documented as a domain shift limitation.

7 Conclusion

This project successfully implements an end-to-end apple orchard monitoring system integrating real-time object detection, fruit instance segmentation, and dual explainability with quantitative validation. The system achieves strong detection performance (mAP@0.5 = 0.987), efficient segmentation via SAM-enhanced training, and interactive visualization through a web-based FastAPI + Streamlit interface.

Future work includes expanding the dataset for better domain generalization, pixel-level defect segmentation when annotations become available, and edge deployment via ONNX/TensorRT conversion.

Technologies used: Python, PyTorch, Ultralytics YOLOv8, segmentation-models-pytorch, FastAPI, Streamlit, Docker, OpenCV, NumPy.

Training platform: Google Colab (NVIDIA T4 GPU).